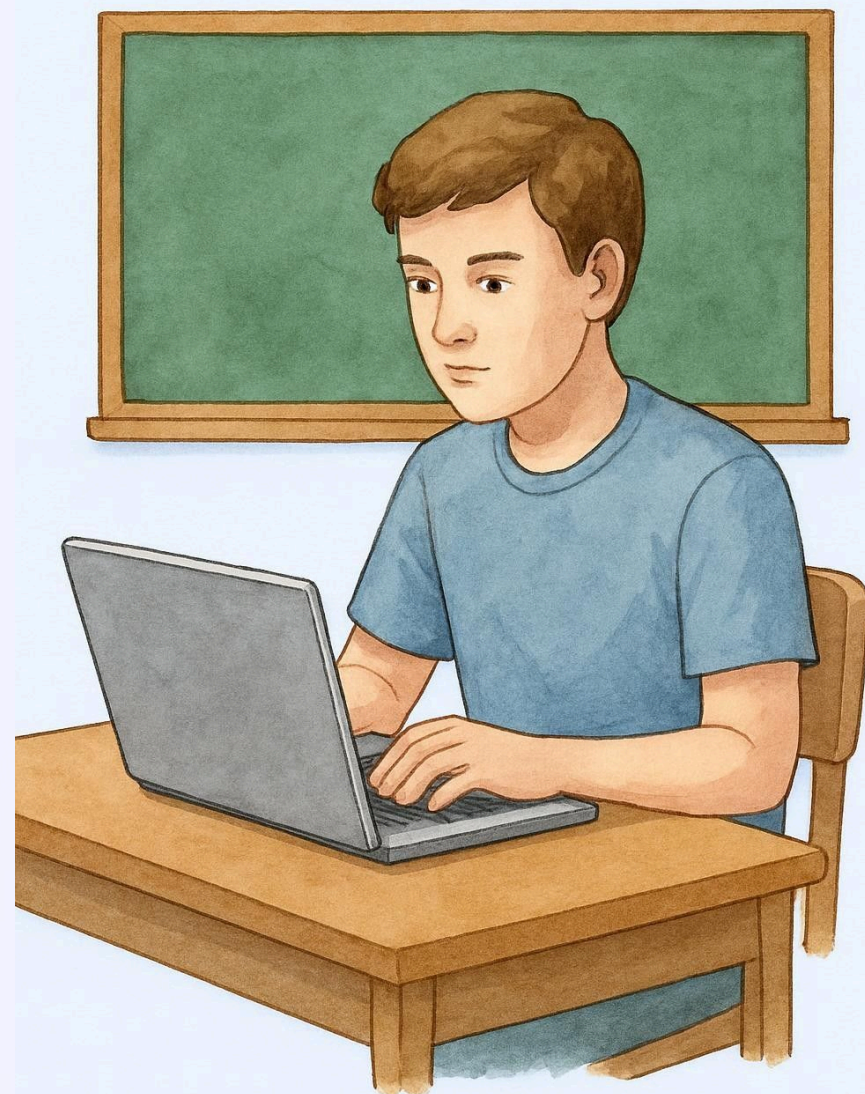


Aula 13 - Atividade em Sala

Desenvolvimento para Ambientes Móveis · Faculdade COTEMIG · 2026.2

AULA 13

ATIVIDADE PRÁTICA



O que faremos hoje

A aula de hoje é mão na massa. Vamos consolidar tudo o que vimos nas últimas três aulas em um único app funcional – do consumo de API à navegação entre telas.

Revisar

Retomar os conceitos das Aulas 10, 11 e 12 e ver como eles se conectam na prática.

Completar

Resolver todos os TO-DOs do arquivo esqueleto `main.dart` fornecido.

Entregar

Ter o app rodando com lista, detalhe, loading e tratamento de erro.

O que você já sabe usar

Cada aula anterior construiu uma peça. Hoje você monta o quebra-cabeça completo.

1

Aula 10

`Navigator.push` e `pop` – navegação entre telas.

2

Aula 11

`Future`, estado de loading e tratamento com `try/catch`.

3

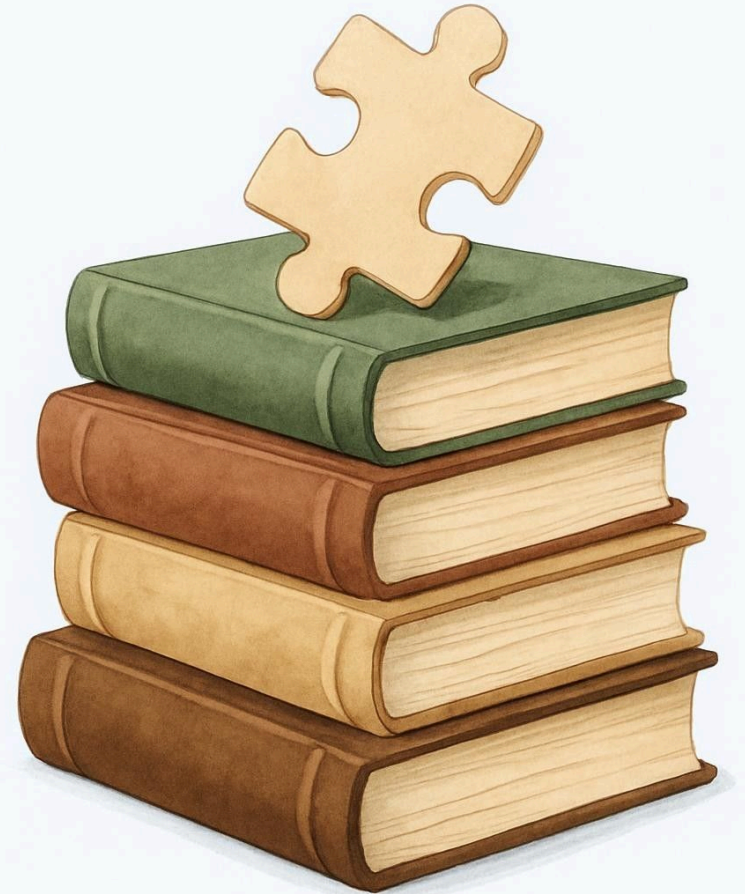
Aula 12

`http.get`, `model class`, `fromJson` e exibição em lista.

4

Aula 13

Junta tudo – app completo, do fetch ao detalhe.



A API que usaremos

Vamos consumir a **Rick and Morty API** – gratuita, sem autenticação e perfeita para praticar.

Endpoint principal:

GET <https://rickandmortyapi.com/api/character>

⚠️ Atenção: os dados dos personagens estão dentro da chave **results**, não na raiz do JSON. Não esqueça disso no **fromjson**!

Estrutura da resposta

```
{  
  "info": { ... },  
  "results": [  
    {  
      "id": 1,  
      "name": "Rick Sanchez",  
      "status": "Alive",  
      "species": "Human",  
      "image": "https://..."  
    },  
    ...  
  ]  
}
```

O Lab de Hoje

Você receberá o arquivo `main.dart` com a estrutura base já criada. Sua missão é localizar cada `// TODO` e implementar o código correspondente. O layout é **livre** use a criatividade para montar a interface do jeito que quiser, desde que a funcionalidade esteja correta.

📁 Arquivo

`main.dart` esqueleto fornecido pelo professor.

🎨 Layout livre

A UI é sua. Use `ListTile`, `Card`, `GridView` o que preferir.

✅ Foco

O que será avaliado é a **lógica**: `fetch`, `parse`, estados e navegação.



Os TO-DOs - Passo a Passo

Os TODOs estão numerados no arquivo. Siga a ordem – cada um depende do anterior.

1 Fazer o GET

Use `http.get(Uri.parse(url))` para chamar o endpoint da API.

2 Verificar o statusCode

Cheque se a resposta foi 200. Caso contrário, lance uma exceção.

3 Extrair results e parsear o JSON

Decodifique o body e acesse `data['results']` para chegar à lista de personagens.

4 Mapear com fromJson

Use `.map((e) => Character.fromJson(e)).toList()` para converter os dados.

5 Exibir: loading / lista / erro

Gerencie o estado com `FutureBuilder` ou `setState` – trate os três casos.

6 Navegar para o detalhe

No toque do item, use `Navigator.push` passando o objeto `Character` selecionado.

Entrega da Aula

Ao final da aula, seu app deve estar **funcionando** com todos os critérios abaixo. A entrega é feita diretamente ao professor, com o app rodando no emulador ou dispositivo.



Lista na tela

Os personagens da API aparecem numa lista scrollável após o carregamento.



Toque abre detalhe

Ao tocar em um item, a tela de detalhe do personagem é aberta com `Navigator.push`.



Erro tratado

Se a API falhar ou a conexão cair, o app exibe uma mensagem de erro – sem travar.



Layout do seu jeito

A interface reflete a sua criatividade. Surpreenda, a originalidade é bem-vinda.

📄 Dúvidas? Conte, conosco. Bom código! 🚀